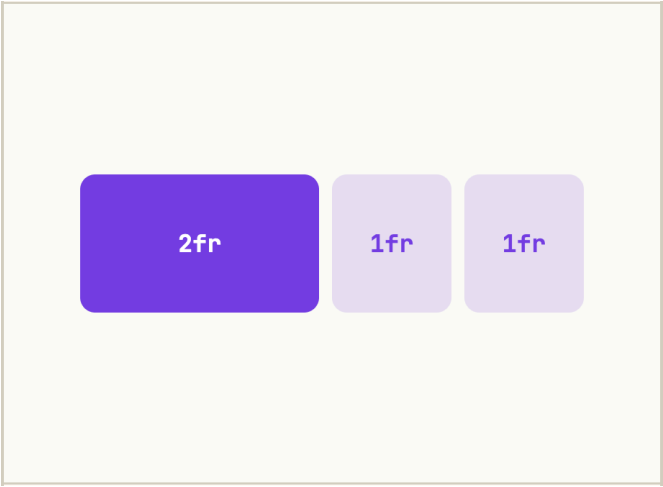
Q1 — Grid Fractions Which CSS creates this layout?



- A `grid-template-columns: 1fr 1fr 1fr;`
- B ✅ `grid-template-columns: 2fr 1fr 1fr;`
- C `grid-template-columns: repeat(3, 1fr);`
- D `grid-template-columns: 200px auto auto;`

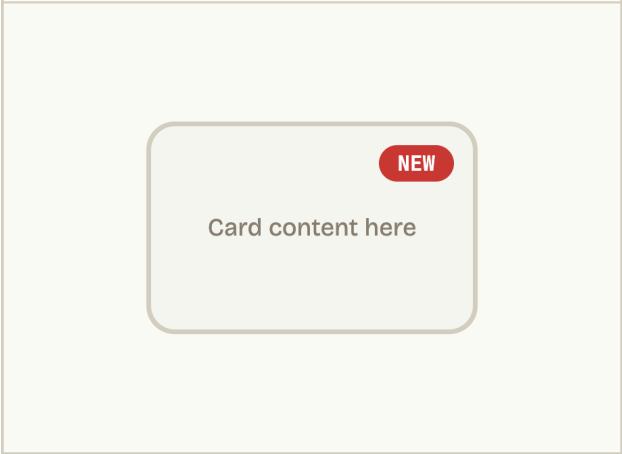
Q2 — CSS Variable Image: Code shown on screen

```
:root { --brand-color: #7c6aff; }
.button { background: var(--brand-color); }
```

What color is the button background?

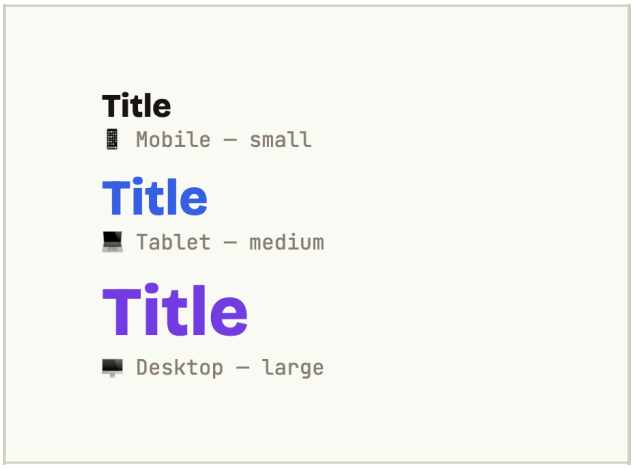
- A No color — var() doesn't work here
- B ✅ `#7c6aff` (purple)
- C The color of the :root element
- D CSS error — invalid syntax

Q3 — Badge Position Which CSS on the badge pins it to the card corner?



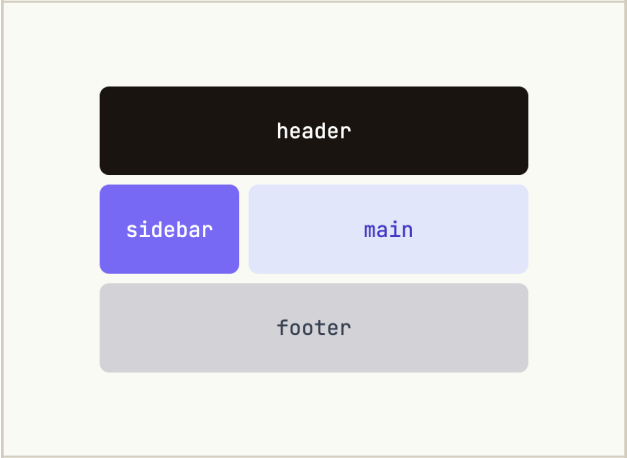
- A `position: fixed; top: 0; right: 0;`
- B `position: static; top: 0; right: 0;`
- C ✅ `position: absolute; top: 0; right: 0;`
- D `position: sticky; top: 0; right: 0;`

Q4 — Clamp Which CSS makes the font size scale fluidly between a min and max?



- A `font-size: min(1rem, 3rem);`
- B `font-size: between(1rem, 3rem);`
- C `font-size: scale(1rem, 3vw, 3rem);`
- D ✅ `font-size: clamp(1rem, 3vw, 3rem);`

Q5 — Grid Area Which CSS defines this layout using named grid areas?



- A ✅

```
grid-template-areas:
  "header header"
  "sidebar main"
  "footer footer";
```

- B

```
grid-template-areas:
  "header header"
  "sidebar main"
  "footer footer";
```

- C

```
grid-areas: header sidebar main footer;
```

- D

```
grid-template-columns:
  "header" "sidebar" "main" "footer";
```

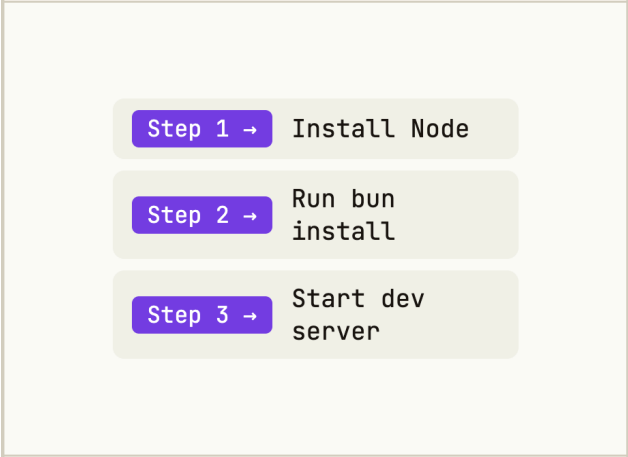
Q6 — Custom Property Fallback

```
.box {
  background: var(--missing-color, coral);
}
```

`--missing-color` is not defined anywhere. What color is the box?

- A Transparent — variable is missing
- B Black — CSS default
- C CSS error — crashes silently
- D ✅ Coral — fallback value is used

Q7 — Counter Which CSS generates automatic counters like this?



- A

```
li::before { content: "Step"; }
```

- B

```
li { list-style: step-counter; }
```

- C

```
li { list-style: step-counter; }
```

- D ✅

```
ol { counter-reset: step; }
li::before {
  counter-increment: step;
  content: "Step " counter(step) " →";
}
```

Q8 — Logical Properties Which CSS property automatically flips for right-to-left languages?

- A `margin-left: 16px;`
- B `padding-left: 16px;`
- C `border-left: 2px solid;`
- D ✅ `margin-inline-start: 16px;`

Q9 — Container Query Which CSS makes a component respond to its PARENT container size instead of the screen size?

- A `@media (min-width: 600px) { .card { flex-direction: row; } }`
- B `@parent (min-width: 600px) { .card { flex-direction: row; } }`
- C `@responsive (min-width: 600px) { .card { flex-direction: row; } }`
- D ✅

```
.wrapper { container-type: inline-size; }
@container (min-width: 600px) {
  .card { flex-direction: row; }
}
```

Q10 — Cascade Layer

```
@layer base {
  button { background: grey; }
}
@layer theme {
  button { background: blue; }
}
```

Which layer's color wins?

- A Grey — `base` is more specific
- B Grey — `base` is declared first
- C They cancel each other out — no color
- D ✅ Blue — later layers always win over earlier ones